Introduction to Helmfile

Declarative multi-chart Kubernetes deployment

DevOps Pro 2025 · Aliaksandr Tsimokhau

Agenda

1. Introduction

What is Helmfile and why it matters · what's new in v1.4/v1.5

2. Structure

`helmfile.yaml` — releases, environments, values

3. Advanced features

Labels, env vars, secrets management

4. Best practices

Templates, layering, DRY patterns

5. Remote charts & values from git

go-getter URLs, OCI, sub-helmfiles, lock files

6. Commands & workflow

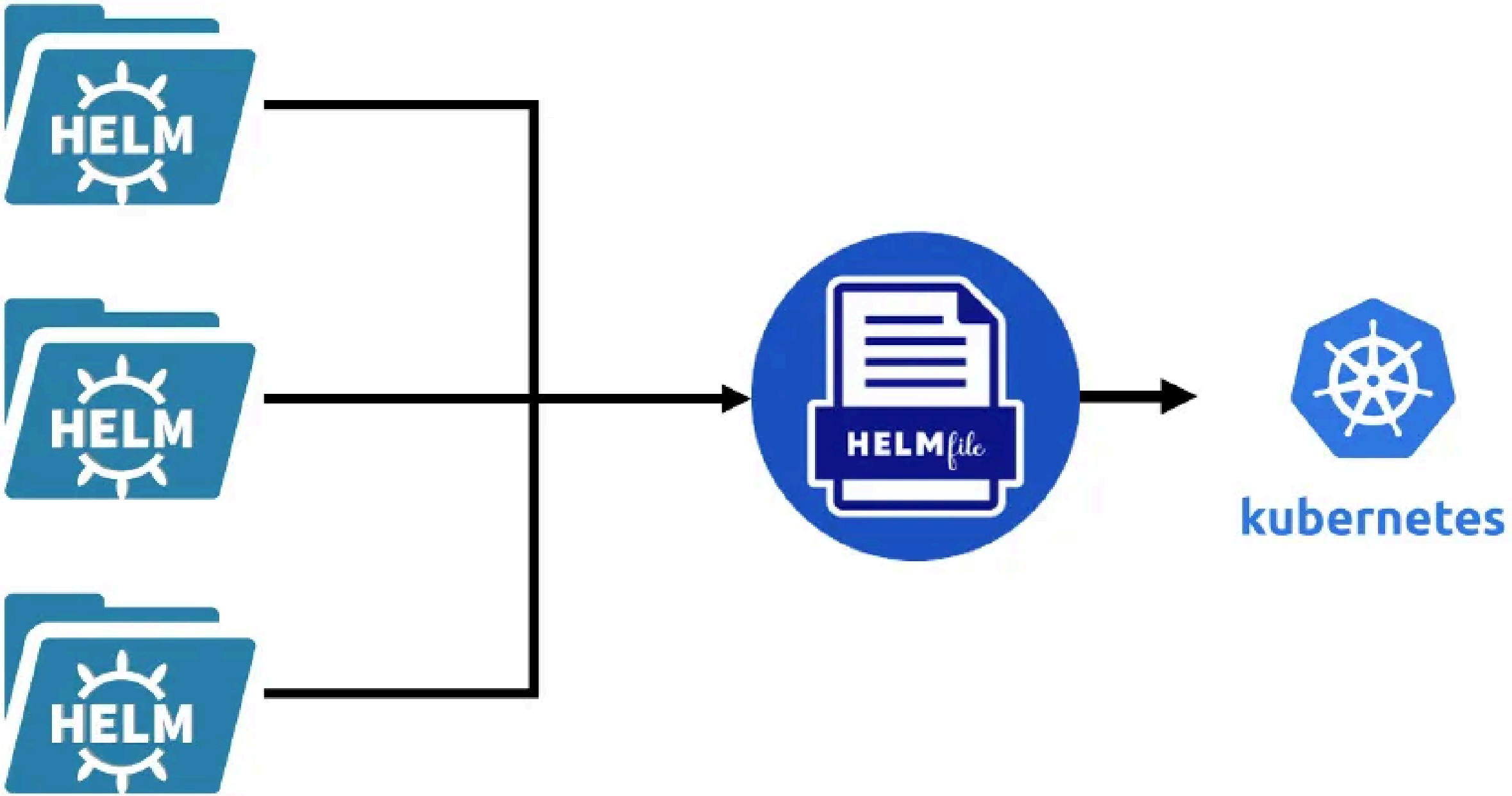
Day-to-day usage and wrap-up

Part 1: Introduction

What Helmfile is and why your team needs it

What is Helmfile?

- **Helmfile** is a declarative spec for deploying Helm charts
- Manages multiple Helm charts as code
- Simplifies complex Kubernetes deployments


HELM
HELM
HELM
HELMfile
kubernetes

Helm vs. Helmfile — where each fits

Feature	Helm	Helmfile
Purpose	Package manager for K8s charts	Declarative management of multiple releases
Release management	Individual releases	Multiple releases in one file
Environment support	Limited	Multiple environments with custom values
Secrets management	No built-in	Integrates with SOPS
Use case	Simple / single charts	Complex / multi-chart, multi-env

Why use Helmfile?

- **Centralized management**: manage all Helm charts and environments in a single file
- **Consistency**: ensures repeatable, version-controlled deployments
- **Simplicity**: reduces manual steps and complexity
- **Automation**: integrates easily with CI/CD pipelines
- **Environment support**: handles multiple environments without duplication
- **Secret management**: securely manages sensitive data with tools like SOPS

Key features

- **Declarative YAML**: define releases, values, and environments
- **Environment support**: manage different configs for dev, staging, prod
- **Secret management**: integrates with SOPS for secrets
- **Remote sources**: pull charts, values, and sub-helmfiles from git / HTTP / OCI

What's new in Helmfile v1.4 / v1.5

Recent releases (2026) added several quality-of-life features:

- **Kubedog integration** (v1.4): real-time tracking of Deployments, StatefulSets, DaemonSets, Jobs with log streaming
- `helmfile create` subcommand (v1.5): scaffold a new project skeleton in one command
- `HELMFILE_*` **env vars** (v1.5): defaults for `--kube-context`, `--namespace`, and other global flags
- `defaultInherit` (v1.5): every release auto-inherits a named template — no more repetitive `inherit:` blocks
- **Per-environment lock files**: `lockFilePath: .helmfile.{{` {{ .Environment.Name }} `}}.lock` — pin versions per env
- **OCI dependencies**: add OCI charts as `dependencies:` of an ad-hoc release

Part 2: Structure

Anatomy of `helmfile.yaml`

Basic structure of `helmfile.yaml`

The three main building blocks:

- **Releases**: define the Helm charts to deploy, their names, and versions
- **Environments**: customize settings for different environments
- **Values**: specify custom configuration values for your charts

`helmfile.yaml` — example

```yaml
environments:
    staging:
        values:
            - values-staging.yaml
    production:
        values:
            - values-production.yaml

releases:
    - name: my-app
        chart: stable/my-app
        namespace: default
        values:
            - common-values.yaml

    - name: my-database
        chart: stable/mysql
        namespace: default
        values:
            - common-values.yaml
            - database-values.yaml
```

Environments in Helmfile

- **Environments** allow different configurations for each deployment scenario
- In the example:
  - **staging** uses `values-staging.yaml`
  - **production** uses `values-production.yaml`
- Separation keeps configurations organized and reduces duplication

Releases in Helmfile

- **Releases** specify which Helm charts to deploy and how to configure them
- Example defines two releases:
  - **my-app**: deploys `stable/my-app` with settings from `common-values.yaml`
  - **my-database**: deploys `stable/mysql` with `common-values.yaml` and extra configs from `database-values.yaml`

What happens during deployment?

When you run `helmfile sync`:

1. Helmfile reads `helmfile.yaml`
2. Applies the correct values for the chosen environment
3. Deploys all defined charts as specified

The three commands you'll see throughout

Before we go deeper — the everyday loop. (Full reference in Part 6.)

Command	What it does
`helmfile template`	Render manifests locally — no cluster touched
`helmfile diff`	Preview what `sync` / `apply` would change
`helmfile apply`	`diff` + deploy **only if** there are changes

> `helmfile sync` force-runs `helm upgrade` on every release; `apply` is the safer default.

Part 3: Advanced features

Labels, environments, env vars, and secrets

Labels in Helmfile

- **Labels** are tags attached to Helm releases
- Help group and manage releases efficiently
- Use labels to target specific releases when running Helmfile commands

```
releases:
    - name: frontend
      chart: stable/frontend
      labels:
          env: production

    - name: backend
      chart: stable/backend
      labels:
          env: staging
```

Using labels with `--selector`

- **Sync only production releases:**

```
helmfile --selector env=production sync
```

- **Sync only staging releases:**

```
helmfile --selector env=staging sync
```

Environments — advanced config

- **Environments** define settings for different deployment scenarios
- Avoids duplication by keeping environment-specific configs in one file

```
environments:
    staging:
        values:
            - values-staging.yaml
    production:
        values:
            - values-production.yaml

releases:
    - name: frontend
      chart: stable/frontend
      namespace: default
    - name: backend
      chart: stable/backend
      namespace: default
```

Deploying to specific environments

- **Deploy to staging:**

```
helmfile --environment staging sync
```

- **Deploy to production:**

```
helmfile --environment production sync
```

Environment variables in Helmfile

- **Environment variables** allow dynamic values in `helmfile.yaml`
- Useful for injecting secrets or configuration at runtime

```
releases:
    - name: my-app
        chart: stable/my-app
        namespace: {{ requiredEnv "NAMESPACE" }}
        values:
            - replicas: {{ env "REPLICA_COUNT" | default "2" }}
```

Using environment variables

- **Set variables manually:**

```
export NAMESPACE=production
export REPLICA_COUNT=5
helmfile sync
```

- **Or load from a `.env` file:**

```
source .env
helmfile sync
```

Secrets management in Helmfile

- Securely manage sensitive data (API keys, passwords) using **SOPS**
- Helmfile can decrypt secrets at deployment time

```
sops --encrypt --output secrets.yaml <<EOF
db_password: my-secret-password
api_key: my-api-key
EOF
```

```
releases:
    - name: my-app
      chart: stable/my-app
      values:
          - values.yaml
          - secrets.yaml
```

Part 4: Best practices

DRY patterns, templates, and layering

`.Values` in Helmfile vs. Helm

- Both Helm and Helmfile use `.Values` in templates
- In Helmfile, `.Values` refers to **environment values**; in Helm it refers to **chart values**
- Helmfile provides `.StateValues` as an alias for its own `.Values` to avoid confusion

```
app:
    project: {{ .Environment.Name }}-{{ .StateValues.project }}
```

Handling missing keys and defaults

- Helmfile fails if you reference a missing key in environment values
- `default` — fallback when the key exists but is empty
- `get` — reads a missing key without failing; its second argument is the fallback

```
{{ .Values.eventApi.replicas | default 1 }}       # fails if eventApi is missing
{{ .Values | get "eventApi.replicas" nil }}       # missing → nil, no failure
{{ .Values | get "eventApi.replicas" 1 }}         # missing → 1
```

Reducing repetition: release templates

- Large projects repeat fields like `namespace`, `chart`, `values`, and `secrets`
- Use **Release Templates** to DRY up configuration

```
templates:
    default:
        chart: stable/{{`{{ .Release.Name }}`}}
        namespace: kube-system
        missingFileHandler: Warn
        values:
            - config/{{`{{ .Release.Name }}`}}/values.yaml
            - config/{{`{{ .Release.Name }}`}}/{{`{{ .Environment.Name }}`}}.yaml
        secrets:
            - config/{{`{{ .Release.Name }}`}}/secrets.yaml
            - config/{{`{{ .Release.Name }}`}}/{{`{{ .Environment.Name }}`}}-secrets.yaml

releases:
    - name: kubernetes-dashboard
        version: 0.10.0
        inherit:
            - template: default
```

Release template features

Templates support:

- Basic fields: `name`, `namespace`, `chart`, `version`
- Boolean fields: `installed`, `wait`, `verify`
- Templated fields: `installedTemplate`, `waitTemplate`
- `setTemplate`, `valuesTemplate`, and `secrets`
- Inline values

```
setTemplate:
    - name: '{{`{{ .Release.Name }}`}}'
      values: '{{`{{ .Release.Namespace }}`}}'
```

Layering state files

- Use **Layering** to share common configuration across multiple Helmfiles
- Extract shared parts (like environments) into separate files

```
# helmfile.yaml
bases:
    - environments.yaml

releases:
    - name: metricbeat
        chart: stable/metricbeat
    - name: myapp
        chart: mychart

# environments.yaml
environments:
    development:
    production:
```

Merging arrays in layers

- Arrays (like `releases`) are **not merged** across layers
- The last defined array **overrides** previous ones

```
# Layer 1
releases:
    - name: metricbeat
      chart: stable/metricbeat
---
# Layer 2
releases:
    - name: myapp
      chart: mychart
```

- Result: only `myapp` remains
- **Workaround**: use YAML anchors or Go templates to import shared releases

Layering state template files

- Use **state template files** with Go templating for advanced DRY patterns
- Each `---`-separated part is a template rendered in sequence

```
# helmfile.yaml.gotmpl
bases:
    - myenv.yaml
---
bases:
    - mydefaults.yaml.gotmpl
---
releases:
    - name: test1
      chart: mychart-{{ .Values.myname }}
      values:
          - replicaCount: 1
            image:
                repository: "nginx"
                tag: "latest"
```

Re-using environment state in sub-Helmfiles

- Load environment state once and pass it to sub-Helmfiles

```
environments:
    stage:
        values:
            - env/stage.yaml
    prod:
        values:
            - env/prod.yaml
---
helmfiles:
    - path: releases/myrelease/helmfile.yaml
        values:
            - {{ toYaml .Values | nindent 4 }}
```

Sub-Helmfile — inherited values

```
releases:
    - name: mychart-{{ .Values.myrelease.myname }}
      installed: {{ .Values | get "myrelease.enabled" false }}
      chart: mychart
      version: {{ .Values.myrelease.version }}
      labels:
        chart: mychart
      values:
        - values.yaml.gotmpl
```

Summary — Best Practices takeaways

- Use `.StateValues` to distinguish Helmfile values from chart values
- Handle missing keys with `default` and `get`
- Reduce repetition with Release Templates
- Layer state files for shared configuration
- Be aware of array overriding in layers
- Use state template files for advanced DRY patterns
- Pass environment state to sub-Helmfiles for modularity

Part 5: Remote charts & values from git

Pull charts, values, and sub-helmfiles directly from remote sources

The problem: shared infra across teams

- Multiple teams reuse the **same** chart (e.g. company-wide `ingress`, `monitoring`)
- Copy-pasting into every repo → drift, security holes, manual upgrades
- Local-only `helmfile.yaml` forces teams to vendor charts

Goal: point at a single source of truth (git repo, OCI registry) and pin a version

Where remote URLs are accepted

Helmfile accepts go-getter URLs in **four** places:

Where	Purpose
`releases[].chart`	Load a Helm chart from git / OCI / HTTP
`releases[].values`	Pull a values file from a remote source
`helmfiles[].path`	Compose a sub-helmfile hosted elsewhere
`bases[]`	Layer in remote shared configuration

go-getter URL syntax

Helmfile uses **hashicorp/go-getter** to fetch remote resources.

General format:

```
<protocol>::<url>@<subpath>?<query>
```

Common protocols and parameters:

- `git::` — clone a git repo (supports https / ssh)
- `@subpath` — pick a sub-directory inside the repo
- `?ref=` — pin to a branch, tag, or commit SHA
- `?sshkey=` — pass an SSH private key for private repos
- `oci://` — pull an OCI artifact (Helm chart in OCI registry)
- Plain HTTPS — fetch a `.tar.gz` or single file

Example 1 — chart from a git repository

```yaml
repositories:
  - name: bitnami
    url: https://charts.bitnami.com/bitnami

releases:
  - name: shared-ingress
    chart: git::https://github.com/myorg/helm-charts.git@charts/ingress?ref=v1.4.0
    namespace: ingress-system
    values:
      - values/ingress.yaml

  - name: feature-app
    chart: git::ssh://git@github.com/myorg/feature-app.git@deploy/chart?ref=main
    namespace: apps
```

- `@charts/ingress` — chart lives inside `charts/ingress/` of the repo
- `?ref=v1.4.0` — **pinned** to a tag; `main` would track the branch HEAD

Example 2 — values file from git

```yaml
environments:
  production:
    values:
      - git::https://github.com/myorg/platform-config.git@envs/prod.yaml?ref=v2025.06
      - git::https://github.com/myorg/platform-config.git@common/defaults.yaml?ref=v2025.06

releases:
  - name: api
    chart: ./charts/api
    values:
      - https://config.example.com/api/values-prod.yaml
      - secrets.yaml
```

- Values can come from **git** (with ref pinning) **or plain HTTPS**
- Combine with local files — Helmfile merges in the order listed (last wins)

Example 3 — sub-helmfile from git

```
# Root helmfile.yaml — orchestrates platform components

environments:
  production:
    values:
      - env/prod.yaml

---
helmfiles:
  - path: git::https://github.com/myorg/platform-helmfiles.git@monitoring/helmfile.yaml?ref=v3.2.0
    values:
      - {{ toYaml .Values | nindent 8 }}

  - path: git::https://github.com/myorg/platform-helmfiles.git@logging/helmfile.yaml?ref=v3.2.0
    values:
      - {{ toYaml .Values | nindent 8 }}

  - path: ./services/helmfile.yaml
```

Example 4 — OCI registry charts

OCI is the **modern way** to distribute Helm charts (Helm 3.8+):

```
repositories:
  - name: ghcr-myorg
    url: oci://ghcr.io/myorg/charts
    oci: true

releases:
  - name: api
    chart: oci://ghcr.io/myorg/charts/api
    version: 1.6.2
    namespace: apps

  - name: worker
    chart: oci://registry.gitlab.com/myorg/charts/worker
    version: 2.0.0
```

- Works with **GHCR, GitLab Registry, ECR, Harbor, Artifactory**
- `version:` field works just like a normal Helm repo
- Since v0.146.0, OCI charts can also be `dependencies:`

Authentication for private repos

- **HTTPS + token** — embed PAT in URL or use git credential helper:

```
chart: git::https://oauth2:${GIT_TOKEN}@github.com/myorg/repo.git@chart?ref=v1.0
```

- **SSH** — rely on `ssh-agent` or pass key via `?sshkey=`:

```
chart: git::ssh://git@github.com/myorg/repo.git@chart?ref=main
```

- **OCI** — log in once: `helm registry login ghcr.io -u USER -p $TOKEN`
- In CI: inject token from secrets, use `${...}` env-var expansion in URLs

Caveats and tips

- **Always pin** `?ref=` — without it you track branch HEAD; one bad merge upstream breaks every consumer
- **Caching**: go-getter caches under `~/.cache/helmfile/` — clear it if you suspect stale content
- **Lock files**: `helmfile deps` writes `helmfile.lock`; combine with per-env lockfile to ramp versions gradually
- `helmfile fetch --write-output` (v1.5+) — pre-download all remote artefacts for air-gapped deployments
- **Diff first**: `helmfile diff` shows exactly what changes when you bump a `?ref=`
- **Sub-path syntax**: `@dir/inside/repo` — note the `@`, not `/`; missing it = clone-then-no-chart-found

Part 6: Commands & workflow

Day-to-day Helmfile usage

Common Helmfile commands

Command	Purpose
`helmfile template`	Render chart templates without deploying
`helmfile diff`	Preview changes before applying
`helmfile sync`	Install or upgrade all releases
`helmfile apply`	Apply changes with interactive approval
`helmfile destroy`	Delete all releases

Workflow example

1. Write `helmfile.yaml` with releases
2. Run `helmfile template` to verify rendered manifests
3. Run `helmfile diff` to preview changes
4. Run `helmfile apply` to deploy with confirmation

Benefits

- **Consistency**: same config for all environments
- **Automation**: integrates with CI/CD pipelines
- **Scalability**: manage many charts easily
- **Auditability**: git history = full deployment audit trail

Helmfile streamlines complex Kubernetes deployments

Ideal for teams managing multiple Helm charts and environments

Questions?

References

- Helmfile documentation
- Helmfile GitHub (current home) — formerly `roboll/helmfile`
- Helmfile releases
- hashicorp/go-getter — URL syntax reference
- Helm OCI registry support
- What is Helmfile? — Devtron Blog